

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA1201	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	U.Prabhavathi	Reg.No.:	192472067
Department:	CSE-AI	Date:	29-08-26

ANNEXURE A

Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I U.Prabhavathi/192472067 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:U.Prabhavathi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

I/O Organization and Data Transfer Algorithms

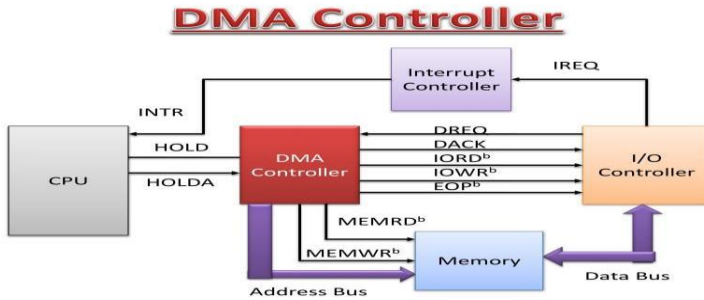
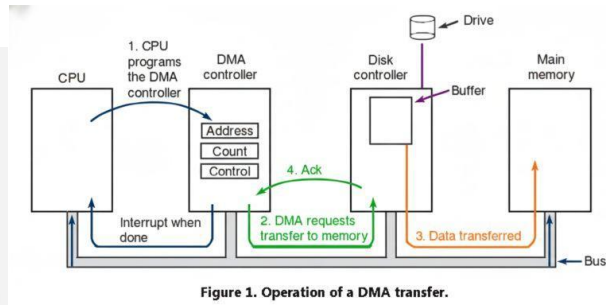
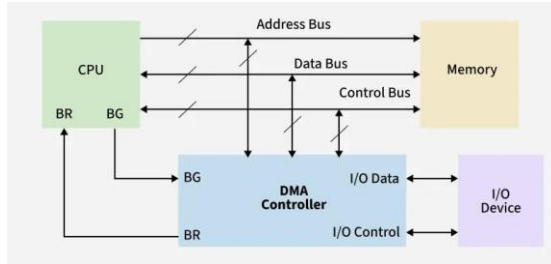
1. DMA-Based Data Transfer Algorithm

1.1 Introduction

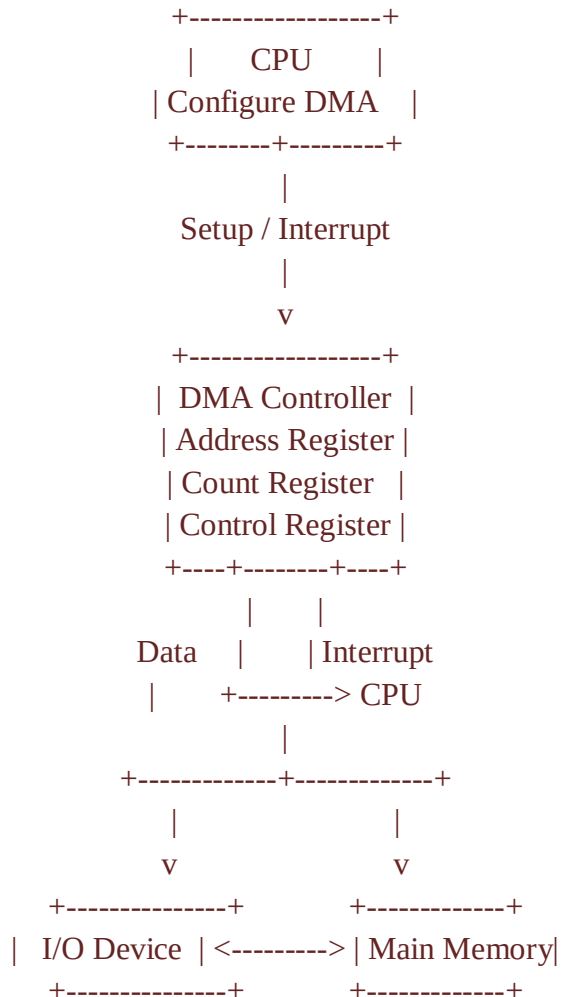
Direct Memory Access (DMA) allows an I/O device to transfer data directly between the device and main memory without requiring the CPU to move every byte or word.

The CPU mainly performs initial configuration and receives an interrupt after the transfer is completed. This reduces CPU involvement and improves overall system performance.

Architecture



5.



DMA Transfer Steps

Step	Operation	CPU Involvement
1	CPU initializes DMA	High
2	CPU specifies memory address	High
3	CPU specifies transfer size	High
4	DMA requests bus	Low
5	DMA transfers data	None
6	DMA updates address/count	None
7	Transfer completes	None
8	DMA generates interrupt	Low
9	CPU handles completion	Low

Pseudocode

ALGORITHM DMA_IO_TO_MEMORY

INPUT:

IO_Device
Memory_Address
Transfer_Size

BEGIN

Initialize DMA_Controller

DMA_Controller.Source $\leftarrow$ IO_Device
DMA_Controller.Destination $\leftarrow$ Memory_Address
DMA_Controller.Count $\leftarrow$ Transfer_Size
DMA_Controller.Mode $\leftarrow$ IO_TO_MEMORY
DMA_Controller.Interrupt_Enable $\leftarrow$ TRUE

Enable DMA_Controller

Start IO_Device

WHILE DMA_Controller.Status $\neq$ COMPLETE DO

CPU performs other tasks

IF DMA_Controller.Status = ERROR THEN

```
    Disable DMA_Controller
    Report transfer error
    RETURN FAILURE
END IF
```

```
END WHILE
```

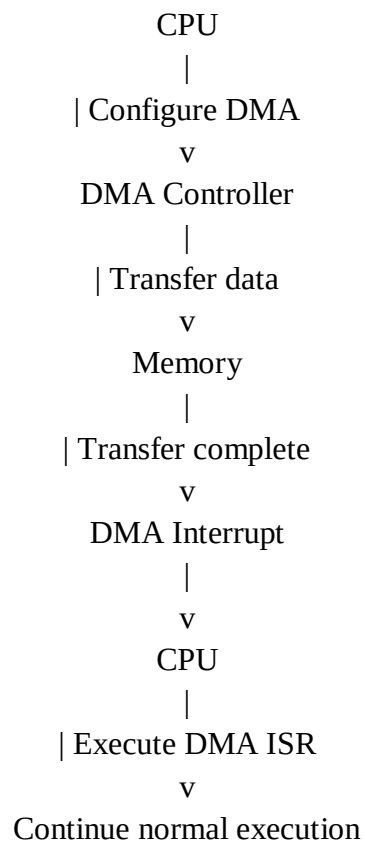
```
WAIT FOR DMA_INTERRUPT
```

```
IF DMA_Controller.Status = COMPLETE THEN
    Clear DMA interrupt flag
    Verify transfer status
    Notify application
    RETURN SUCCESS
END IF
```

```
END
```

Interrupt Handling

Instead of continuously checking the DMA controller:



Advantages

- Minimizes CPU involvement.
- Faster transfer for large blocks.
- CPU can execute another task during transfer.
- Reduces interrupt frequency compared with byte-by-byte transfer.
- Suitable for disks, network interfaces, audio/video devices, etc.

2. Vectored Interrupt Handling System

2.1 Introduction

An interrupt is a signal generated by an I/O device requesting CPU attention.

In a vectored interrupt system, every interrupt source has an associated interrupt vector. The vector identifies the appropriate Interrupt Service Routine (ISR).

For example:

Keyboard $\rightarrow$ Vector 1 $\rightarrow$ Keyboard ISR

Network $\rightarrow$ Vector 2 $\rightarrow$ Network ISR

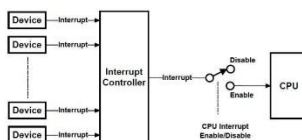
Disk $\rightarrow$ Vector 3 $\rightarrow$ Disk ISR

Timer $\rightarrow$ Vector 4 $\rightarrow$ Timer ISR

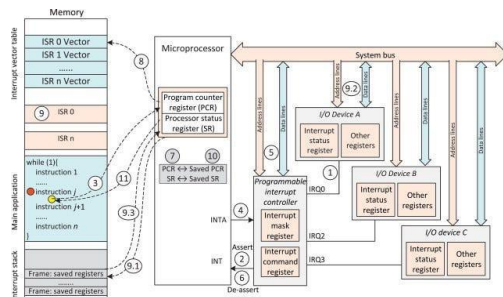
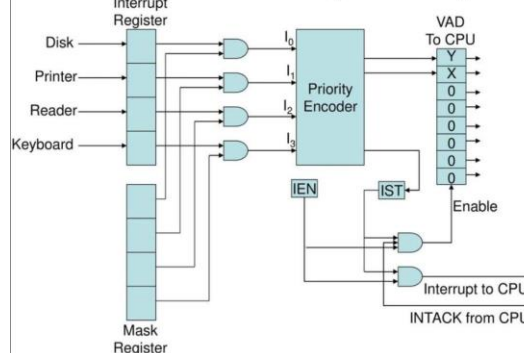
Architecture

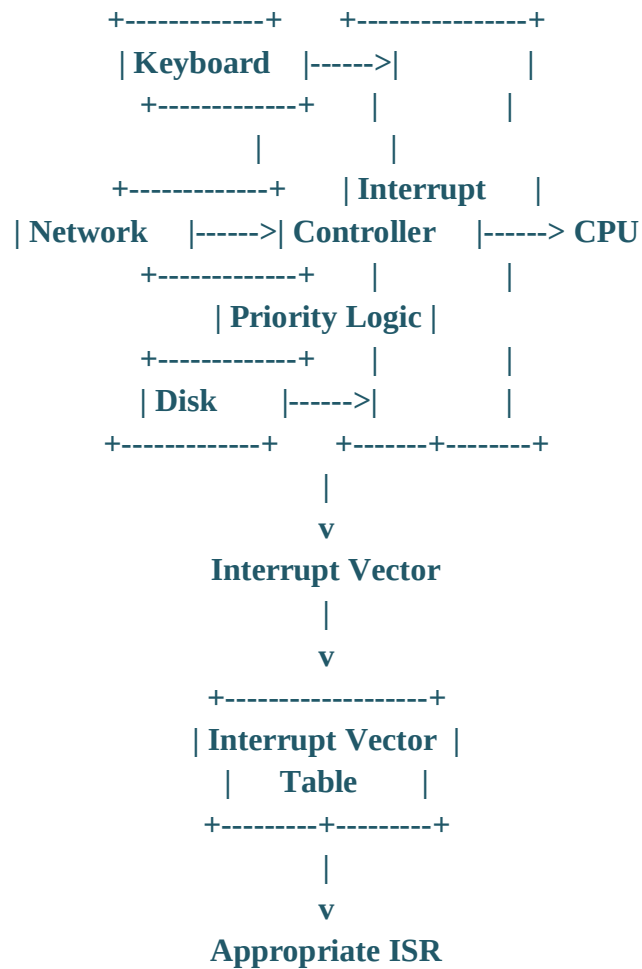
Interrupt Handling

- Interrupt controller handles many issues including providing the address of the required ISR in many cases
- Two models:
 - All interrupts vector to a single interrupt handler
 - Each interrupt vector directly to its own ISR



Parallel Priority Interrupt





Priority Example

Suppose:

Device	Priority	Vector
Network	1	20
Disk	2	21
Keyboard	3	22
Printer	4	23

Here **1 = highest priority**.

If Network and Keyboard generate interrupts simultaneously, the Network interrupt is handled first.

Pseudocode

ALGORITHM VECTORED_INTERRUPT_HANDLER

INPUT:

Interrupt_Request[1...N]

BEGIN

Save CPU registers

Disable lower-priority interrupts temporarily

WHILE Interrupt_Request exists DO

Highest_Priority $\leftarrow$ Find_Highest_Priority(
Interrupt_Request)

Vector_Number $\leftarrow$ Get_Vector(
Highest_Priority)

ISR_Address $\leftarrow$ Vector_Table[Vector_Number]

Acknowledge_Interrupt(Highest_Priority)

Call ISR_Address

Clear Interrupt_Request[Highest_Priority]

END WHILE

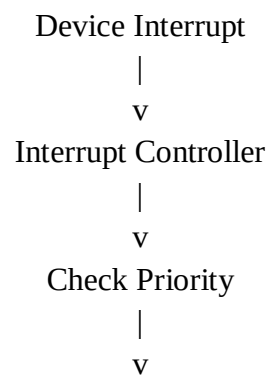
Restore CPU registers

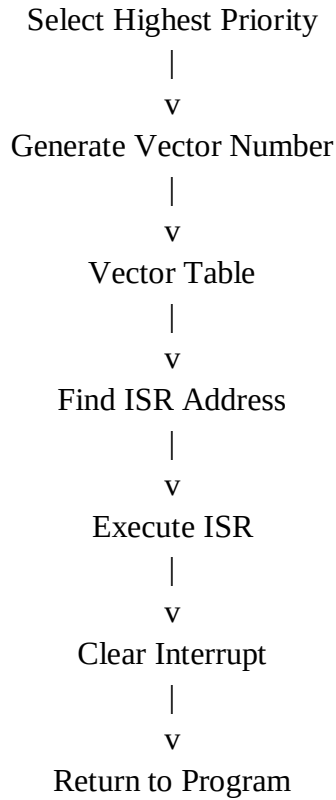
Enable interrupts

RETURN

END

Interrupt Dispatch Process





Advantages

- Fast interrupt dispatch.
- Supports multiple devices.
- Priority-based handling.
- Reduces time spent identifying interrupt sources.
- Suitable for real-time systems.

3. Dynamic Selection of Memory-Mapped I/O and I/O-Mapped I/O

3.1 Introduction

There are two major techniques for addressing I/O devices.

Memory-Mapped I/O

I/O registers are assigned addresses within the normal memory address space.

CPU

|

+---- Memory Address ----> RAM

|

+---- I/O Address -----> Device Register

The CPU can use normal load/store instructions.

I/O-Mapped I/O

I/O devices have a separate address space.

Memory Address Space

|
+---- RAM

I/O Address Space

|
+---- I/O Devices

Special I/O instructions are generally used.

Comparison

Feature	Memory-Mapped I/O	I/O-Mapped I/O
Address space	Shared with memory	Separate
Instructions	Normal load/store	Special I/O instructions
Programming	Simple	More specialized
Address space usage	Uses memory address space	Separate I/O space
Flexibility	High	Moderate
Common consideration	Fast register access	Dedicated I/O addressing

Dynamic Selection Criteria

The system can consider:

- Latency
- Bandwidth
- Transfer size
- Device type
- CPU overhead
- Address-space availability
- Number of transactions

Decision Table

Requirement	Preferred Method
Very low latency	Memory-mapped I/O
High-frequency register access	Memory-mapped I/O
Large data transfers	DMA generally preferred
Dedicated I/O address space desired	I/O-mapped I/O
Simple load/store access	Memory-mapped I/O
Limited memory address space	I/O-mapped I/O

Pseudocode

ALGORITHM SELECT_IO_MAPPING

INPUT:

Latency_Requirement
Bandwidth_Requirement
Address_Space_Available
Transfer_Size
CPU_Overhead

OUTPUT:

Selected_IO_Method

BEGIN

IF Latency_Requirement = HIGH THEN

 IF Address_Space_Available = TRUE THEN

 Selected_IO_Method ← MEMORY_MAPPED_IO

 ELSE

 Selected_IO_Method ← IO_MAPPED_IO

 END IF

ELSE IF Bandwidth_Requirement = HIGH THEN

 IF Transfer_Size > LARGE_TRANSFER_THRESHOLD THEN

 Enable DMA

```

IF Address_Space_Available = TRUE THEN
    Selected_IO_Method ← MEMORY_MAPPED_IO + DMA
ELSE
    Selected_IO_Method ← IO_MAPPED_IO + DMA
END IF

ELSE
    Selected_IO_Method ← MEMORY_MAPPED_IO
END IF

ELSE IF CPU_Overhead = HIGH THEN

    Enable DMA

    Selected_IO_Method ← MEMORY_MAPPED_IO + DMA

ELSE

    Selected_IO_Method ← IO_MAPPED_IO

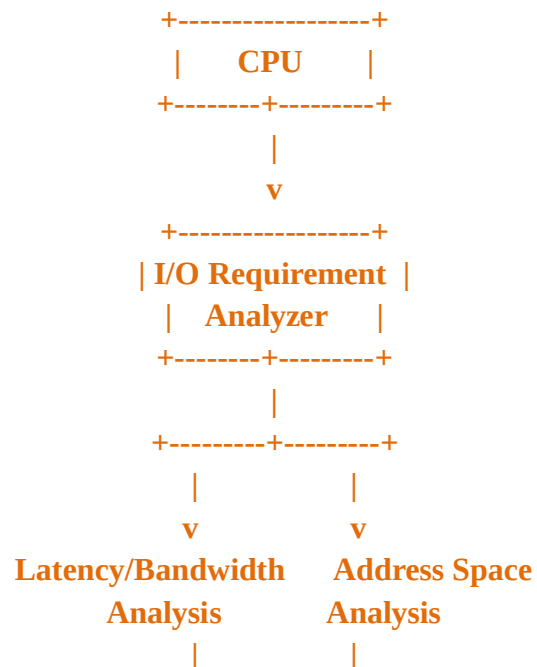
END IF

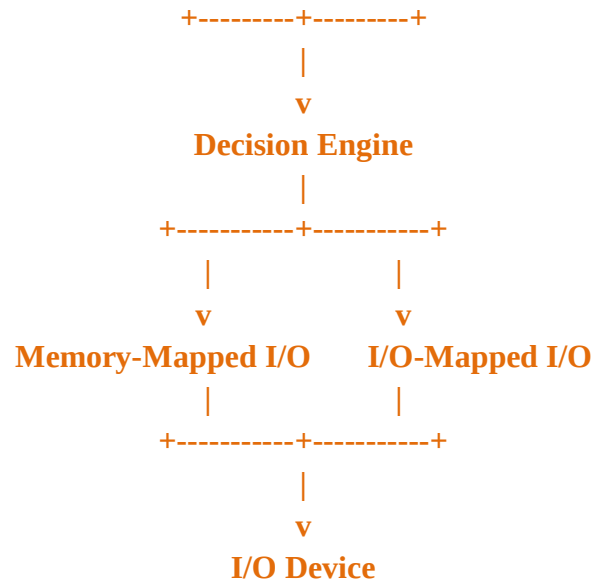
RETURN Selected_IO_Method

END

```

Dynamic Architecture





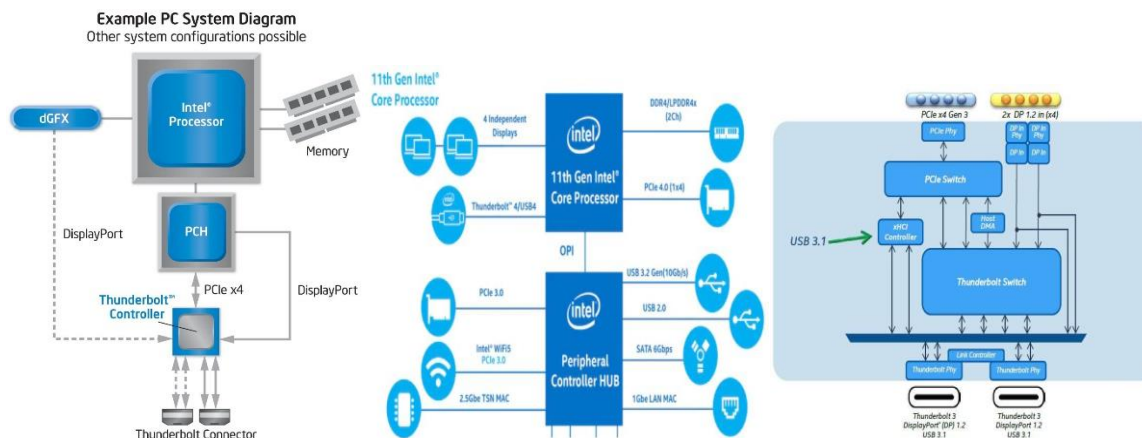
4. Bus Arbitration Algorithm for PCIe, USB and Thunderbolt

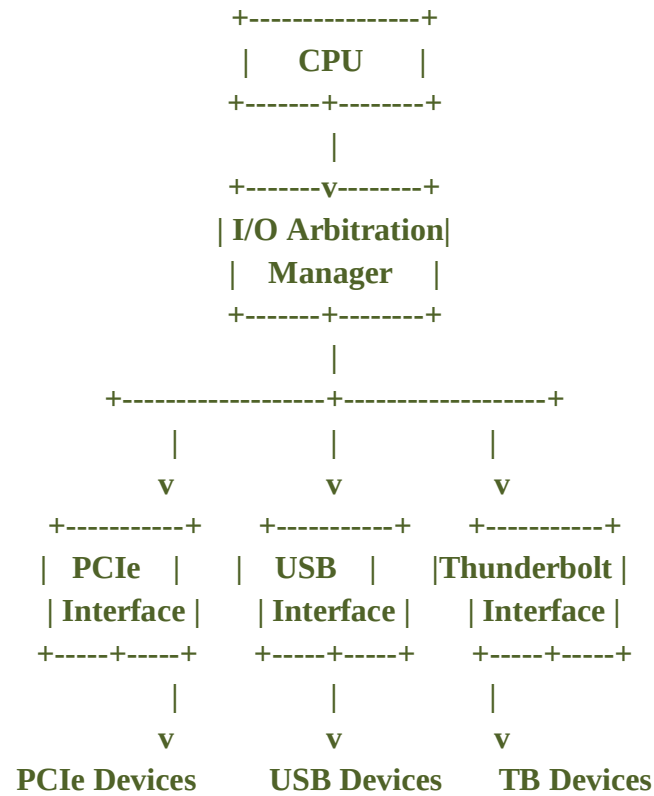
4.1 Introduction

Bus arbitration determines which device or requester gets access to a shared communication resource when multiple devices request access simultaneously.

Modern interfaces such as PCIe, USB, and Thunderbolt do not all use a single traditional shared parallel bus. They use different architectures and arbitration mechanisms. Therefore, the pseudocode below models a generic resource-arbitration layer that can manage requests arriving from these interfaces.

Architecture





Arbitration Factors

The algorithm considers:

1. Priority
2. Latency requirement
3. Bandwidth requirement
4. Request age
5. Interface type
6. Fairness

Example Priority

Request	Interface	Priority
Network transfer	PCIe	5
High-speed storage	Thunderbolt	4
USB camera	USB	3
Keyboard	USB	1

Higher value = higher priority.

Pseudocode

ALGORITHM MULTI_INTERFACE_BUS_ARBITRATION

INPUT:

Request_Queue

BEGIN

Initialize PCIe_Interface

Initialize USB_Interface

Initialize Thunderbolt_Interface

WHILE System_Running = TRUE DO

Collect all pending requests

FOR each Request in Request_Queue DO

Determine Interface_Type

IF Interface_Type = PCIe THEN

Base_Priority $\leftarrow$ PCIe_Priority

ELSE IF Interface_Type = USB THEN

Base_Priority $\leftarrow$ USB_Priority

ELSE IF Interface_Type = THUNDERBOLT THEN

Base_Priority $\leftarrow$ Thunderbolt_Priority

END IF

Calculate Priority_Score

Priority_Score $\leftarrow$

Base_Priority

+ Latency_Weight

+ Bandwidth_Weight

+ Aging_Factor

END FOR

Selected_Request $\leftarrow$ Request with

Highest Priority_Score

IF Resource_Available(Selected_Request) THEN

Grant_Access(Selected_Request)

Start_Transfer(Selected_Request)

WAIT until Transfer_Complete

Release_Resource(Selected_Request)

Remove Selected_Request from Request_Queue

ELSE

Update_Request_Status()

END IF

END WHILE

END

Fairness Mechanism

Aging can prevent starvation.

$\text{Aging_Factor} = \text{Current_Time} - \text{Request_Arrival_Time}$

Thus, an old low-priority request gradually receives a higher effective priority.

Arbitration Flow

Device Requests

|

v

Collect Requests

|

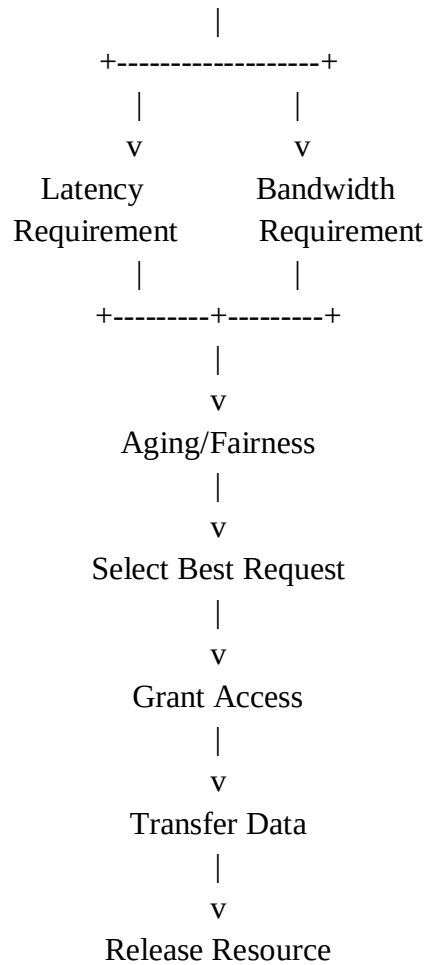
v

Identify Interface

|

v

Calculate Priority



Advantages

- Supports different interface types.
- Prioritizes latency-sensitive traffic.
- Prevents starvation.
- Improves bandwidth utilization.
- Can support quality-of-service requirements.

5. Hybrid I/O Communication Mechanism

5.1 Introduction

A hybrid I/O mechanism combines:

Polling

The CPU periodically checks the status of a device.

Suitable for:

- Low-speed devices
- Simple devices
- Devices with predictable activity

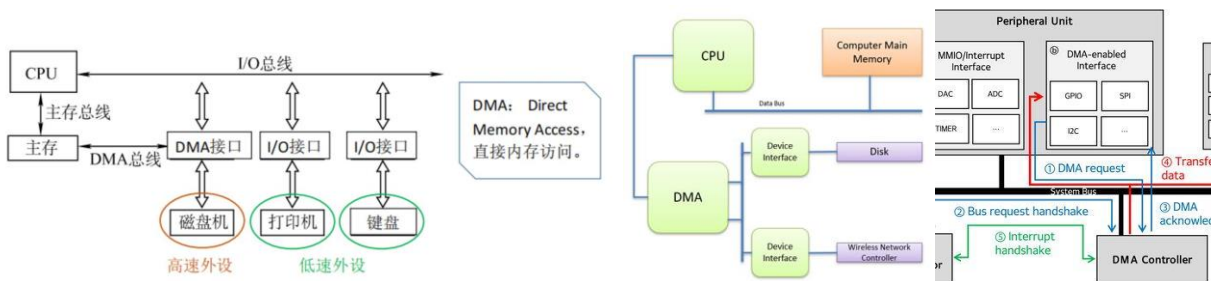
DMA + Interrupts

For high-speed devices, DMA transfers large amounts of data while an interrupt informs the CPU after completion.

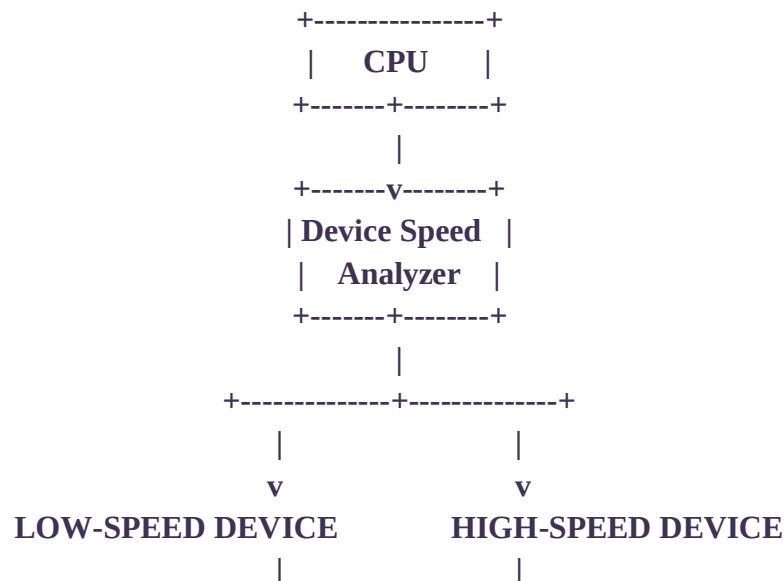
Suitable for:

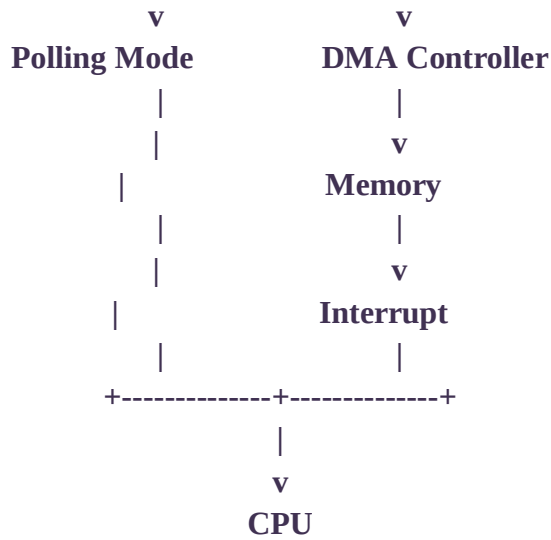
- SSD/storage
- Network interfaces
- High-speed cameras
- Audio/video streams

Architecture



4





Device Classification

Device	Speed	Communication Method
Keyboard	Low	Polling
Mouse	Low/medium	Polling
Temperature sensor	Low	Polling
Printer	Low	Polling/Interrupt
SSD	High	DMA + Interrupt
Network card	High	DMA + Interrupt
Camera	High	DMA + Interrupt
Video device	High	DMA + Interrupt

Pseudocode

ALGORITHM HYBRID_IO_COMMUNICATION

INPUT:

Device_List

BEGIN

FOR each Device in Device_List DO

 Speed ← Measure_Device_Speed(Device)

 Bandwidth ← Estimate_Bandwidth(Device)

```

Data_Size ← Determine_Transfer_Size(Device)

IF Speed = LOW AND
  Bandwidth < LOW_BANDWIDTH_THRESHOLD THEN

  Device.Mode ← POLLING

ELSE

  Device.Mode ← DMA_INTERRUPT

END IF

END FOR

WHILE System_Running = TRUE DO

  FOR each Device in Device_List DO

    IF Device.Mode = POLLING THEN

      Status ← Check_Device_Status(Device)

      IF Status = DATA_AVAILABLE THEN

        Data ← Read_Data(Device)

        Store_Data_In_Memory(Data)

      END IF

    ELSE IF Device.Mode = DMA_INTERRUPT THEN

      Configure_DMA(Device)

      DMA_Source ← Device
      DMA_Destination ← Memory
      DMA_Size ← Device.Data_Size

      Start_DMA_Transfer(Device)

```

```

        Continue_CPU_Work()

    END IF

END FOR

IF DMA_INTERRUPT_RECEIVED THEN

    Device ← Identify_DMA_Source()

    IF Transfer_Status(Device) = SUCCESS THEN

        Clear_DMA_Interrupt(Device)

        Process_Transferred_Data(Device)

    ELSE

        Handle_DMA_Error(Device)

    END IF

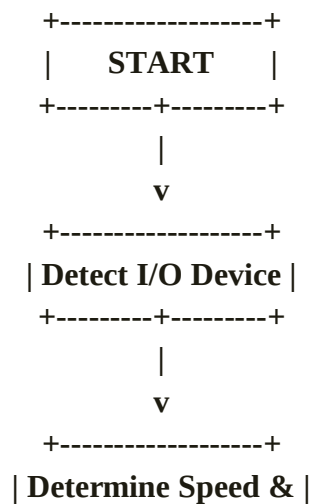
END IF

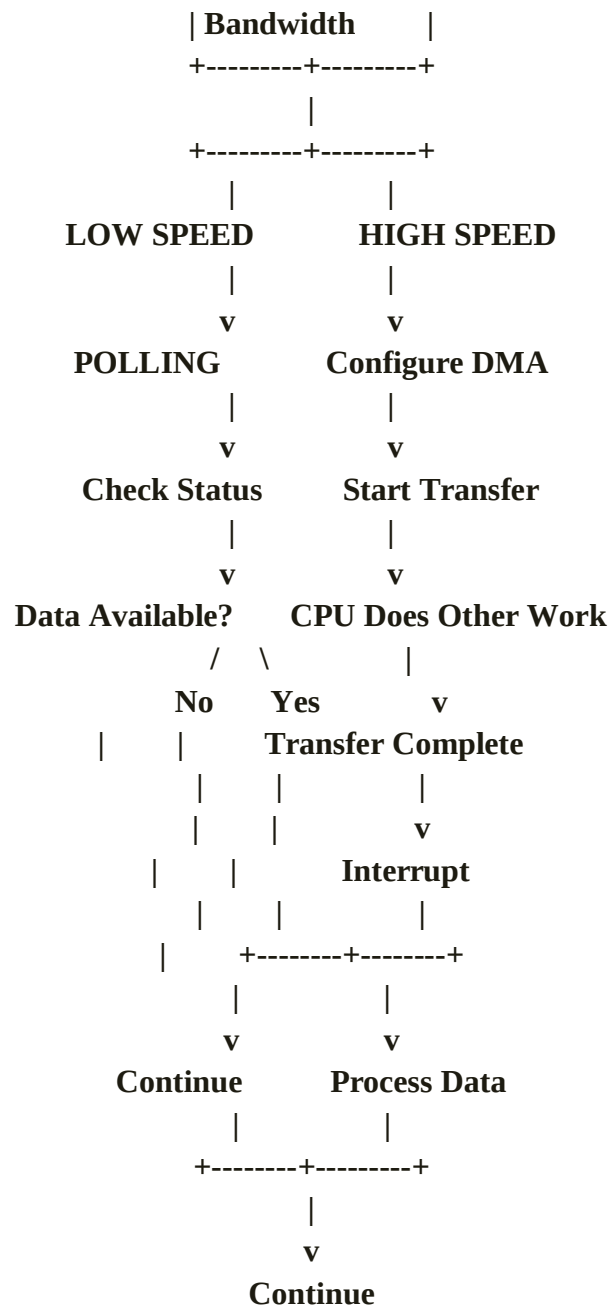
END WHILE

END

```

Overall Hybrid Working





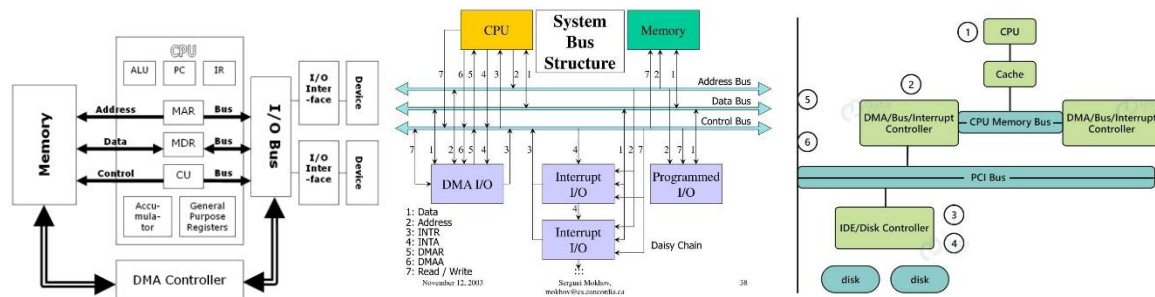
Comparison of All Five Algorithms

No.	Algorithm	Main Purpose	CPU Involvement	Best Use
1	DMA Transfer	Device → Memory transfer	Very Low	High-volume data
2	Vectored Interrupt	Handle multiple interrupts	Low	Multiple I/O devices
3	Dynamic I/O Mapping	Select I/O addressing method	Medium	Different I/O requirements
4	Bus Arbitration	Manage competing requests	Low/Medium	Multiple interfaces

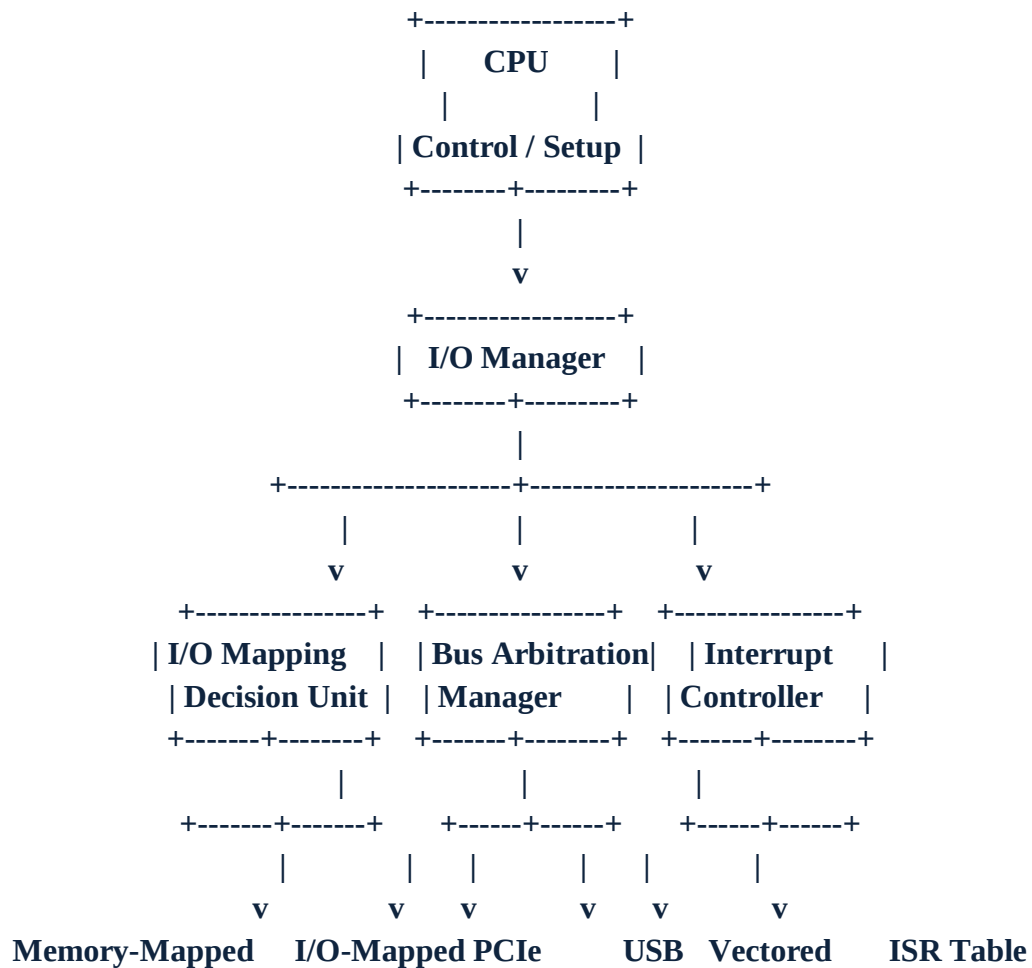
No.	Algorithm	Main Purpose	CPU Involvement	Best Use
5	Hybrid I/O	Select polling or DMA	Low	Mixed-speed devices

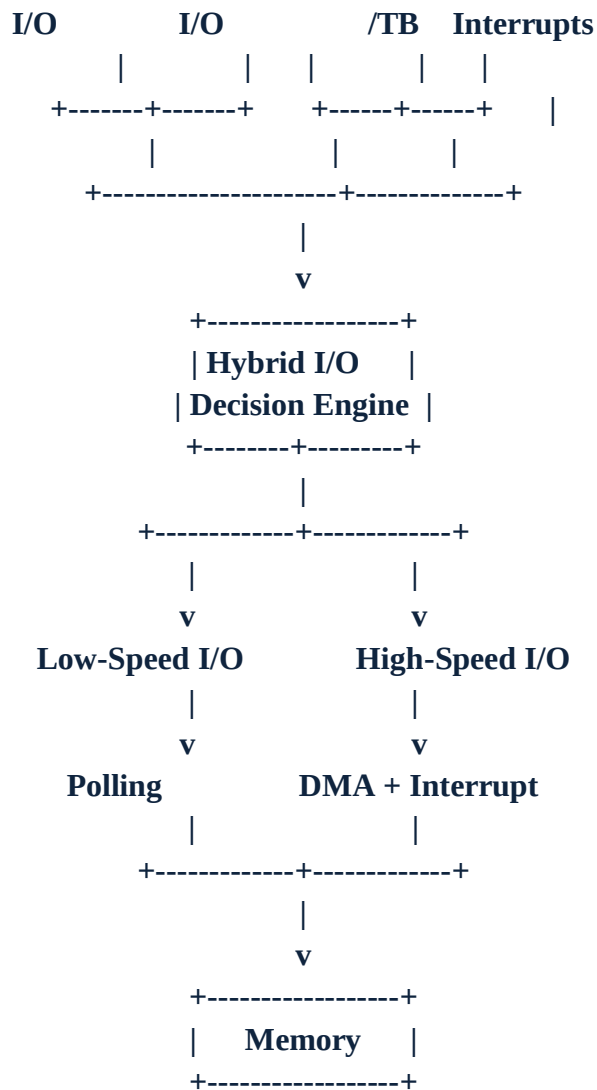
Integrated System Architecture

The five concepts can be combined into one complete I/O architecture:



6





Overall Working Sequence

The complete system operates as follows:

Step 1: The CPU identifies the I/O device and its requirements.

Step 2: The I/O manager determines whether memory-mapped I/O or I/O-mapped I/O is appropriate.

Step 3: The system determines device speed.

Step 4: Low-speed devices are handled using polling.

Step 5: High-speed devices are configured for DMA.

Step 6: If several devices request resources simultaneously, the arbitration manager selects the request based on priority, latency, bandwidth, and aging.

Step 7: After high-speed DMA transfer completes, the device/DMA controller generates an interrupt.

Step 8: The interrupt controller determines the highest-priority interrupt and obtains its vector.

Step 9: The corresponding Interrupt Service Routine (ISR) is executed.

Step 10: The CPU resumes its normal execution.

Key Points for Examination

Concept	Key Point
DMA	Transfers data between I/O and memory with minimal CPU intervention
Interrupt	Allows device to notify CPU when attention is required
Vectored Interrupt	Vector directly identifies the required ISR
Memory-Mapped I/O	I/O registers occupy normal memory address space
I/O-Mapped I/O	I/O devices use a separate I/O address space
Bus Arbitration	Decides which requester gets access to a communication resource
Polling	CPU repeatedly checks device status
Hybrid I/O	Uses polling for low-speed devices and DMA + interrupts for high-speed devices
Priority	Determines which request/interrupt is handled first
Aging	Prevents low-priority requests from waiting indefinitely

Conclusion

The proposed I/O system minimizes CPU overhead by combining DMA, vectored interrupts, dynamic I/O mapping, intelligent arbitration, and hybrid communication. Low-speed devices can be handled efficiently through polling, while high-speed devices use DMA to transfer large data blocks directly to memory. Vectored interrupts provide fast service-routine selection, and arbitration ensures fair and efficient allocation of communication resources. This architecture is therefore suitable as a conceptual model for modern high-performance computer I/O systems.